Here's a **clear step-by-step implementation on Ubuntu (Terminal)** for your full Git practical 👇

✅ Step 0: Install & Configure Git (First Time Only)

```shell
sudo apt update
sudo apt install git -y

git --version
```

Set your identity:

```shell
git config --global user.name "Your Name"
git config --global user.email "your-email@example.com"
```

✅ a. Initialize a Git Repository for Data Science Project

1. Create project folder

```shell
mkdir DataScienceProject
cd DataScienceProject
```

2. Initialize Git

```shell
git init
```

👉 This creates a hidden `.git` folder (repository initialized)

✅ b. Push it to GitHub

1. Create repository on GitHub

- Go to GitHub
- Click **New Repository**
- Name: `DataScienceProject`
- Click **Create**

2. Link local repo to GitHub

```shell
git remote add origin
https://github.com/your-username/DataScienceProject.git
```

Check:

```shell
git remote -v
```

✅ c. Create Python file (program1.py)

```shell
nano program1.py
```

👉 Paste this code:

```python
num = int(input("Enter a number: "))

for i in range(1, 11):
    print(num, "x", i, "=", num * i)
```

Save:

- Press `CTRL + O` → Enter
- Press `CTRL + X`

✅ d. Commit & Push Changes

1. Check status

```shell
git status
```

2. Add file

```shell
git add program1.py
```

3. Commit

```shell
git commit -m "Added program1.py for multiplication table"
```

4. Push to GitHub

```shell
git branch -M main
```

```
git push -u origin main
```

👉 Now check your GitHub repo → file should be visible ✅

✅ e. Create New Branch (GIT-EXAM) & Add program2

1. Create branch

```shell
git checkout -b GIT-EXAM
```

👉 Now you are working on new branch

---

2. Create new file

```shell
nano program2.py
```

👉 Paste this code:

```python
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))

if a > b:
    print("Greatest number is:", a)
else:
    print("Greatest number is:", b)
```

Save:

- `CTRL + O` → Enter → `CTRL + X`

---

3. Add, Commit & Push

```shell
git add program2.py
git commit -m "Added program2.py for greatest of two numbers"
git push -u origin GIT-EXAM
```

✅ f. Show Difference Between main and GIT-EXAM Branch

1. Compare branches

```shell
git diff main GIT-EXAM
```

👉 Shows differences between both branches

---

2. View branch list

```shell
git branch
```

---

3. Switch branches

```shell
git checkout main
```

👉 Notice:

- `program2.py` will NOT be present in `main`

Switch back:

```shell
git checkout GIT-EXAM
```

👉 Now `program2.py` appears again

---

4. Compare commits visually

```shell
git log --oneline --graph --all
```

🎯 Final Output You Must Show in Exam

✔ Git initialized
✔ program1.py in main branch
✔ program2.py in GIT-EXAM branch
✔ Both pushed to GitHub
✔ Difference using `git diff`
✔ Branch switching demo

💡 Important Viva Questions (Exam Ready)

Q1. What is Git?
👉 Version control system to track code changes

Q2. Difference between Git & GitHub?
👉 Git = tool, GitHub = cloud platform

Q3. What is a branch?
👉 Separate workspace for development

Q4. Why use branches?
👉 Avoid affecting main code

Q5. What is commit?
👉 Snapshot of changes

If you want, I can also give you **exact viva answers + tricky examiner questions** based on this practical 🔥

2.

Got it 👍 — I'll give you a **clean Ubuntu-only (Terminal) version** exactly how you should perform in exam.

🐧 ✅ Complete Practical Using Ubuntu Terminal

✅ a. Initialize Git Repository

```shell
mkdir DataScienceProject
cd DataScienceProject
git init
```

✅ b. Push to GitHub

1. Create repo on GitHub (manually in browser)

- Name: `DataScienceProject`

2. Connect Ubuntu repo to GitHub

```shell
git remote add origin
https://github.com/your-username/DataScienceProject.git
```

✅ c. Create program1.py (Even/Odd)

```shell
nano program1.py
```

👉 Paste:

```python
num = int(input("Enter a number: "))

if num % 2 == 0:
    print("Number is Even")
else:
    print("Number is Odd")
```

Save:

- `CTRL + O` → Enter
- `CTRL + X`

✅ d. Commit & Push

```shell
git add program1.py
git commit -m "Added even odd program"
git branch -M main
git push -u origin main
```

👉 Check on GitHub → file should be visible ✅

✅ e. Create Branch & program2.py

1. Create branch

```shell
git checkout -b GIT-EXAM
```

---

2. Create file

```shell
nano program2.py
```

👉 Paste:

```python
string = input("Enter a string: ")
print("Reversed string is:", string[::-1])
```

Save:

- `CTRL + O` → Enter
- `CTRL + X`

---

3. Commit & Push

```shell
git add program2.py
git commit -m "Added reverse string program"
git push -u origin GIT-EXAM
```

✅ f. Show Difference Between Branches

1. Difference command

```shell
git diff main GIT-EXAM
```

---

2. Switch branches (VERY IMPORTANT FOR EXAM)

```shell
git checkout main
```

👉 program2.py will disappear

```shell
git checkout GIT-EXAM
```

👉 program2.py will appear again

✅ g. Create Pull Request (IMPORTANT)

👉 **In Ubuntu exam, do like this:**

1. Open browser (Firefox/Chrome)
2. Go to your GitHub repo
3. Click **Compare & Pull Request**
4. Select:
    - base → main
    - compare → GIT-EXAM
5. Click:
    - Create Pull Request
    - Merge Pull Request
    - Confirm Merge

🔄 Sync merged code locally

```shell
git checkout main
git pull origin main
```

🎯 What Examiner Wants to SEE (Very Important)

✔ Terminal commands working
✔ program1.py in main
✔ program2.py in GIT-EXAM
✔ `git diff` output
✔ Branch switching demo
✔ Pull request on GitHub

⚠️ Common Mistakes (Avoid in Exam)

❌ Forgetting `git add`
❌ Wrong branch while coding
❌ Not pushing branch (`git push -u origin GIT-EXAM`)
❌ Not showing difference

💡 Quick Viva Answers (Ubuntu-Based)

Q1. How to create branch?
👉 `git checkout -b branch_name`

Q2. How to switch branch?
👉 `git checkout branch_name`

Q3. How to push code?
👉 `git push origin branch_name`

Q4. How to check changes?
👉 `git diff`

Q5. Why pull request?
👉 To merge code safely after review

If you want, I can also give you **exact viva questions your external examiner will ask (with answers)** — very useful for scoring 🔥

3.

Perfect 👍 this is a **slightly advanced Git practical (with branching + conflicts)**. I'll give you a **clean Ubuntu Terminal step-by-step** so you can perform it confidently in exam.

🐧 ✅ FULL PRACTICAL (Ubuntu Terminal)

✅ a. Initialize Git Repository

```shell
mkdir DataScienceProject
cd DataScienceProject
git init
```

✅ b. Push to GitHub

👉 Create repo on GitHub: `DataScienceProject`

```shell
git remote add origin
https://github.com/your-username/DataScienceProject.git
```

✅ c. Create Branch GIT-EXAM-1 + program2 (Palindrome)

```shell
git checkout -b GIT-EXAM-1
```

Create file:

```shell
nano program2.py
```

👉 Code:

```python
string = input("Enter a string: ")
```

```
if string == string[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

Save → `CTRL+O`, Enter → `CTRL+X`

✅ d. Create program1.py (Squares 1–10)

```shell
nano program1.py
```

👉 Code:

```python
for i in range(1, 11):
    print(i, "square =", i*i)
```

Save file

✅ e. Commit & Push (GIT-EXAM-1)

```shell
git add .
git commit -m "Added palindrome and squares program"
git push -u origin GIT-EXAM-1
```

👉 ✔ Check on GitHub

✅ f. Create Branch GIT-EXAM-2

```shell
git checkout -b GIT-EXAM-2
```

✅ g. Pull Changes from GIT-EXAM-1

```shell
git pull origin GIT-EXAM-1
```

👉 Now both files are available in this branch

✅ h. Create program2.py (Again Palindrome)

```shell
nano program2.py
```

👉 (Modify or rewrite slightly different version)

```python
string = input("Enter string: ")

rev = "".join(reversed(string))

if string == rev:
```

```
    print("Palindrome")
else:
    print("Not Palindrome")
```

✅ i. Modify program1.py (To Create Conflict)

```shell
nano program1.py
```

👉 Modify code like this:

```python
for i in range(1, 11):
    print("Square of", i, "is", i*i)
```

👉 (This change will create conflict later)

✅ j. Commit, Push & Show Differences

```shell
git add .
git commit -m "Modified program1 and updated palindrome logic"
git push -u origin GIT-EXAM-2
```

🔍 Show Differences

1. Compare main (if exists)

```shell
git branch -M main
```

```shell
git diff main GIT-EXAM-1
git diff GIT-EXAM-1 GIT-EXAM-2
```

---

2. Show commit graph

```shell
git log --oneline --graph --all
```

---

3. Switch branches (IMPORTANT DEMO)

```shell
git checkout GIT-EXAM-1
git checkout GIT-EXAM-2
```

⚠️ (IMPORTANT) Conflict Demonstration

👉 Merge GIT-EXAM-2 into GIT-EXAM-1:

```shell
git checkout GIT-EXAM-1
git merge GIT-EXAM-2
```

👉 You may see conflict in `program1.py`

Open file:

```shell
nano program1.py
```

👉 You will see:

```none
<<<<<<< HEAD
(old code)
=======
(new code)
>>>>>>> GIT-EXAM-2
```

👉 Fix manually → keep one version:

```python
for i in range(1, 11):
    print("Square of", i, "is", i*i)
```

Resolve conflict:

```shell
git add program1.py
git commit -m "Resolved merge conflict"
```

✅ k. Create Pull Request (GIT-EXAM-2 → GIT-EXAM-1)

👉 On GitHub:

1. Open repo
2. Click **Compare & Pull Request**
3. Set:

    - base → `GIT-EXAM-1`
    - compare → `GIT-EXAM-2`
4. Click:
    - Create Pull Request
    - Merge Pull Request
    - Confirm Merge

🎯 Final Output (Exam Checklist)

✔ Repo initialized
✔ GIT-EXAM-1 branch created
✔ program1 + program2 added
✔ GIT-EXAM-2 branch created
✔ Pulled changes
✔ Modified code (conflict created)
✔ Differences shown using `git diff`
✔ Conflict resolved
✔ Pull request created

💡 Very Important Viva Questions

Q1. What is merge conflict?

👉 When two branches modify same code → Git cannot decide automatically

Q2. How to resolve conflict?

👉 Edit file manually → `git add` → `git commit`

Q3. Difference between pull & fetch?

👉 Pull = fetch + merge

Q4. Why branching?

👉 Parallel development without affecting main code

Q5. What is HEAD?

👉 Current working branch pointer

If you want, I can give you **1-page revision notes (super fast before exam)** or **expected viva questions your examiner WILL ask from this practical** 🔥

4.

Great 👍 this is a **very important Git practical (with .gitignore)** and commonly asked in exams. I'll give you a **clear Ubuntu Terminal (Git Bash style) step-by-step**.

🐧 ✅ FULL PRACTICAL – Ubuntu Terminal

✅ a. Initialize Git Repository

```shell
mkdir DataScienceProject
cd DataScienceProject
git init
```

✅ b. Push to GitHub

👉 Create repo on GitHub: `DataScienceProject`

```shell
git remote add origin
https://github.com/your-username/DataScienceProject.git
```

✅ c. Create Branch GIT-EXAM-1 + program2 (Palindrome)

```shell
git checkout -b GIT-EXAM-1
```

Create file:

```shell
nano program2.py
```

👉 Code:

```python
string = input("Enter a string: ")

if string == string[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

Save → `CTRL+O` → Enter → `CTRL+X`

✅ d. Create program1.py (Squares 1–10)

```shell
nano program1.py
```

👉 Code:

```python
for i in range(1, 11):
    print(i, "square =", i*i)
```

✅ e. Commit & Push

```shell
git add .
git commit -m "Added palindrome and squares program"
git push -u origin GIT-EXAM-1
```

👉 ✔ Check GitHub → both files visible

✅ f. Create IGNORE Directory + Files

```shell
mkdir IGNORE
cd IGNORE
touch test1.txt test2.txt
cd ..
```

👉 Verify:

```shell
ls IGNORE
```

✅ g. Create .gitignore File

```shell
nano .gitignore
```

👉 Add this line:

```none
IGNORE/
```

Save file

⚠️ Important Step (very important in exam)

If files are already tracked, remove them:

```shell
git rm -r --cached IGNORE
```

✅ h. Commit & Push (.gitignore)

```shell
git add .gitignore
git commit -m "Added .gitignore to ignore IGNORE folder"
git push
```

🎯 Verify IGNORE is NOT pushed

1. Check Git status

```shell
git status
```

👉 IGNORE folder should NOT appear

---

2. Check GitHub

👉 Open repo →
✔ You will see:

- program1.py
- program2.py
- .gitignore

❌ You will NOT see:

- IGNORE folder
- test1.txt
- test2.txt

🧠 Important Concept (Exam Theory)

👉 `.gitignore` is used to:

- Ignore files/folders
- Prevent unnecessary files from being tracked
- Example: logs, temp files, datasets

⚠️ Common Mistakes (VERY IMPORTANT)

❌ Forgetting `git rm --cached`
❌ Writing wrong path in `.gitignore`
❌ Adding files before ignoring

💡 Viva Questions (Exam Ready)

Q1. What is .gitignore?

👉 File used to ignore files/folders in Git

Q2. Why IGNORE folder not pushed?

👉 Because it is listed in `.gitignore`

Q3. What is `--cached`?

👉 Removes files from staging but keeps them locally

Q4. Can we ignore already committed files?

👉 Yes, using `git rm --cached`

Q5. Give example of ignored files?

👉 `.log`, `.env`, `node_modules/`, datasets

🔥 Exam Tip (VERY IMPORTANT)

👉 Always SHOW this:

1. `ls` → folder exists locally
2. GitHub → folder NOT present

👉 This proves `.gitignore` is working ✅

If you want, I can give you **1-page revision cheat sheet for ALL Git practicals (very useful before viva)** 🚀

5.

Ahh got it 👍 — you mean **GitHub Desktop is installed on Ubuntu**, and you must use it (not terminal).
No problem — here is the **correct step-by-step using GitHub Desktop on Ubuntu** 👇

🖥️ ✅ FULL PRACTICAL USING GitHub Desktop (Ubuntu)

✅ a. Initialize Git Repository

1. Open **GitHub Desktop**
2. Click **File** → **New Repository**
3. Enter:
    - Name: `DataScienceProject`
    - Local Path: choose folder
4. Click **Create Repository**

👉 ✔ Repository created locally

✅ b. Push to GitHub

1. Click **Publish Repository** (top-right)
2. Select:
    - Public/Private (any)
3. Click **Publish Repository**

👉 ✔ Now repo is on GitHub

✅ c. Create Branch GIT-EXAM-1 + program2

1. Create Branch

- Click **Current Branch** → **New Branch**
- Name: `GIT-EXAM-1`
- Click **Create Branch**

---

2. Create program2.py

1. Click **Show in File Manager**
2. Create file: `program2.py`

👉 Code:

```python
string = input("Enter a string: ")

if string == string[::-1]:
    print("Palindrome")
else:
    print("Not Palindrome")
```

✅ d. Create program1.py (Squares)

Create file: `program1.py`

```python
for i in range(1, 11):
    print(i, "square =", i*i)
```

✅ e. Commit & Push

1. Go back to GitHub Desktop
2. You will see both files listed

👉 Commit:

- Message: `Added palindrome and squares program`
- Click **Commit to GIT-EXAM-1**

👉 Push:

- Click **Push origin**

👉 ✔ Check GitHub → both files visible

✅ f. Create IGNORE Folder + Files

1. Open project folder
2. Create folder: `IGNORE`
3. Inside it create:
    - `test1.txt`
    - `test2.txt`

✅ g. Create .gitignore File

1. In project folder → Create file `.gitignore`

👉 Add:

```
None
IGNORE/
```

Save file

⚠️ IMPORTANT (Exam Trick)

If IGNORE folder appears in GitHub Desktop:

1. Go to **Repository** → **Open in Terminal**
2. Run:

```shell
git rm -r --cached IGNORE
```

👉 (This step is VERY important for marks)

✅ h. Commit & Push

1. In GitHub Desktop:
   - You will see `.gitignore` changes
2. Commit:
   - Message: `Added gitignore to ignore IGNORE folder`
3. Click **Commit to GIT-EXAM-1**
4. Click **Push origin**

🎯 Verification (SHOW THIS IN EXAM)

1. In File Manager

✔ IGNORE folder exists

---

2. On GitHub website

✔ You will see:

- program1.py
- program2.py
- .gitignore

❌ You will NOT see:

- IGNORE folder
- test1.txt
- test2.txt

🧠 Concept (Viva)

👉 `.gitignore`:

- Prevents files/folders from being tracked
- Used for logs, temp files, datasets

⚠️ Common Mistakes

❌ Not committing before push
❌ Forgetting `.gitignore`
❌ Not removing cached files
❌ Wrong folder name

💡 Viva Questions

Q1. What is GitHub Desktop?

👉 GUI tool to manage Git repositories

Q2. Why use .gitignore?

👉 To avoid pushing unwanted files

Q3. How to verify ignore?

👉 Files present locally but not on GitHub

🔥 Final Exam Tip

👉 Always demonstrate:

- Folder exists locally
- Not visible on GitHub

👉 This gives you full marks ✅

If you want, I can give you **exact viva questions your examiner will ask from this (with answers)** 🔥

6.

Great 👍 this is a **real-world Git workflow practical (clone → fork → branch → PR → conflict resolution)**. I'll explain it clearly for **Ubuntu (Git Bash / Terminal + browser)** step-by-step.

🐧 ✅ FULL PRACTICAL STEPS

✅ a. Clone Repository

```shell
git clone https://github.com/KalyanM45/AI-Project-Gallery.git
cd AI-Project-Gallery
```

👉 This downloads the project locally

✅ b. Demonstrate Branch Creation is Disallowed

👉 Try creating a branch and pushing:

```shell
git checkout -b test-branch
git push origin test-branch
```

❌ You will get error like:

```none
permission denied
```

👉 **Explain in exam:**

- This is **not your repository**
- You don't have write access
- So push/branch creation is restricted

✅ c. Fork the Repository

👉 Go to browser:

- Open repo: https://github.com/KalyanM45/AI-Project-Gallery
- Click **Fork**

👉 Now repo is copied to YOUR GitHub account

✅ d. Create Feature Branch (GIT-EXAM)

1. Clone YOUR forked repo

```shell
git clone https://github.com/your-username/AI-Project-Gallery.git
cd AI-Project-Gallery
```

---

2. Create branch

```shell
git checkout -b GIT-EXAM
```

✅ e. Add / Modify Code

👉 Example: create new file

```shell
nano program.py
```

👉 Add any code:

```python
print("This is my feature update")
```

Save file

Commit & Push

```shell
git add .
git commit -m "Added new feature in GIT-EXAM branch"
git push origin GIT-EXAM
```

✅ f. Submit Pull Request

👉 On GitHub:

1. Open your forked repo
2. Click **Compare & pull request**
3. Set:
    - Base repo: original repo
    - Base branch: `main`
    - Compare: `GIT-EXAM`
4. Click:
    - **Create Pull Request**

👉 ✔ PR submitted

✅ g. Resolve Merge Conflicts (Important)

👉 If conflict occurs:

1. Pull latest changes

```shell
git checkout GIT-EXAM
git pull origin main
```

---

2. If conflict appears

Open file:

```shell
nano filename
```

You'll see:

```none
<<<<<<< HEAD
your code
=======
other code
>>>>>>> main
```

👉 Edit manually → keep correct code

---

3. Resolve conflict

```shell
git add .
git commit -m "Resolved merge conflict"
git push origin GIT-EXAM
```

👉 PR will update automatically ✅

🎯 Final Output (What Examiner Wants)

✔ Clone original repo
✔ Show permission denied

✔ Fork repository
✔ Create new branch
✔ Add/modify code
✔ Push to forked repo
✔ Create pull request
✔ Resolve conflict

🧠 Important Concepts (Viva)

Q1. What is Fork?

👉 Copy of repository to your account

Q2. Difference: Clone vs Fork?

👉 Clone = local copy
👉 Fork = GitHub copy

Q3. Why branch not allowed?

👉 No write permission

Q4. What is Pull Request?

👉 Request to merge changes into original repo

Q5. What is Merge Conflict?

👉 When two people change same code

⚠️ Common Mistakes

❌ Cloning wrong repo (not forked one)
❌ Not pushing branch
❌ Wrong base branch in PR
❌ Not resolving conflict properly

🔥 Exam Tip (VERY IMPORTANT)

👉 ALWAYS say:

> "Since I don't have write access, I forked the repository and created a feature branch to contribute."

👉 This gives you **extra marks** 💯

If you want, I can give you **diagram explanation (fork → branch → PR flow)** or **top tricky viva questions examiner asks from THIS practical** 🔥

7.

Got it 👍 — this is the **GitHub Desktop (GUI) version** of fork + branch + PR workflow. I'll give you **exact exam-ready steps using GitHub Desktop**.

🖥️ ✅ FULL PRACTICAL USING GitHub Desktop

✅ a. Clone Repository

1. Open **GitHub Desktop**
2. Click **File → Clone Repository**
3. Go to **URL tab**

4. Paste:

```
None
https://github.com/KalyanM45/Spam-Email-Detection.git
```

5.
   Choose local path
6. Click **Clone**

👉 ✔ Repository downloaded locally

✅ b. Demonstrate Branch Creation is Disallowed

Try this:

1. Click **Current Branch** → **New Branch**
2. Name: `test-branch`
3. Click **Create Branch**
4. Click **Push origin**

❌ You will get error (permission denied)

👉 **Explain in exam:**

- This is not your repo
- You don't have write access
- So you cannot push changes

✅ c. Fork the Repository

👉 Open in browser:

- Go to original repo
- Click **Fork**

👉 Now it is copied to your GitHub account

✅ d. Create Feature Branch (GIT-EXAM)

1. Clone YOUR fork

1. In GitHub Desktop → **File** → **Clone Repository**
2. Select your forked repo
3. Click **Clone**

---

2. Create branch

- Click **Current Branch** → **New Branch**
- Name: `GIT-EXAM`
- Click **Create Branch**

✅ e. Add / Modify Code

1. Click **Show in File Manager**
2. Create a file (example): `program.py`

👉 Code:

```python
print("Spam Email Detection - Feature Update")
```

Commit & Push

1. Go back to GitHub Desktop
2. You will see changes

👉 Commit:

- Message: `Added new feature`
- Click **Commit to GIT-EXAM**

👉 Push:

- Click **Push origin**

✅ f. Submit Pull Request

1. Click **Create Pull Request** (in GitHub Desktop)
   OR
   Click **View on GitHub**
2. On GitHub:
   - Base repo → original repo
   - Base branch → main
   - Compare → GIT-EXAM
3. Click:
   - **Create Pull Request**

👉 ✔ PR submitted

✅ g. Resolve Merge Conflicts

👉 If conflict occurs:

1. Fetch latest changes

- In GitHub Desktop → Click **Fetch origin**

---

2. Pull updates

- Click **Pull origin**

---

3. If conflict appears

- GitHub Desktop will show **conflicted files**

👉 Click file → edit manually:

```
None
<<<<<<< HEAD
your code
=======
other code
>>>>>>> main
```

👉 Keep correct version and remove markers

---

4. Mark as resolved

- Click **Mark as resolved**

---

5. Commit & Push

- Commit message: `Resolved conflict`
- Click **Commit**
- Click **Push origin**

👉 PR gets updated automatically ✅

🎯 Final Output (Exam Checklist)

✔ Cloned original repo
✔ Showed push denied
✔ Forked repository
✔ Created GIT-EXAM branch
✔ Added code
✔ Committed & pushed
✔ Created pull request
✔ Resolved conflict

🧠 Important Viva Questions

Q1. What is Fork?

👉 Copy of repo to your GitHub account

Q2. Why fork needed?

👉 No direct write access

Q3. What is Pull Request?

👉 Request to merge your changes

Q4. What is conflict?

👉 Same file edited differently

Q5. How resolve conflict in GitHub Desktop?

👉 Edit file → mark resolved → commit

⚠️ Common Mistakes

❌ Not cloning forked repo
❌ Wrong branch while committing
❌ Not pushing before PR
❌ Ignoring conflict resolution

🔥 Exam Tip

👉 Say this line:

> "Since I don't have permission, I forked the repository and created a feature branch."

👉 This gives extra marks 💯

If you want, I can give you **1-page revision of ALL GitHub Desktop practicals + viva questions (super useful before exam)** 🚀

8.

Great 👍 this is a **very important DevOps practical (Jenkins + GitHub CI/CD)**. I'll give you a **clear Ubuntu-based step-by-step** so you can perform AND explain it in exam.

🚀 ✅ JENKINS PRACTICAL (UBUNTU + GITHUB)

✅ a. Create GitHub Repository & Upload Files

1. Go to **GitHub**
2. Create repo: `Jenkins-Project`

Upload files from Ubuntu:

```shell
mkdir JenkinsProject
cd JenkinsProject
git init
git remote add origin
https://github.com/your-username/Jenkins-Project.git
```

Create sample file:

```shell
nano app.py
```

```python
print("Hello from Jenkins Pipeline")
```

Push:

```shell
git add .
git commit -m "Initial commit"
git push -u origin main
```

✅ b. Install & Launch Jenkins

Install Jenkins

```shell
sudo apt update
sudo apt install openjdk-17-jdk -y
```

```shell
wget -q -O - https://pkg.jenkins.io/debian-stable/jenkins.io.key
| sudo apt-key add -
sudo sh -c 'echo deb https://pkg.jenkins.io/debian-stable binary/
> /etc/apt/sources.list.d/jenkins.list'
sudo apt update
sudo apt install jenkins -y
```

Start Jenkins

```shell
sudo systemctl start jenkins
sudo systemctl enable jenkins
```

Open in browser:

👉 http://localhost:8080

Get password:

```shell
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

✅ c. Install Required Plugins

During setup OR later:

Go to:

- Manage Jenkins → Plugins

Install:

- Git Plugin

- GitHub Plugin
- Pipeline Plugin

✅ d. Create New Pipeline Job

1. Dashboard → **New Item**
2. Enter name: `MyPipeline`
3. Select **Pipeline**
4. Click **OK**

✅ e. Configure Pipeline from SCM

1. Scroll to **Pipeline section**
2. Select:
    - Definition → *Pipeline script from SCM*
    - SCM → Git
3. Enter your repo URL:

```
None
https://github.com/your-username/Jenkins-Project.git
```

4. Script Path:

```
None
Jenkinsfile
```

✅ f. Add Jenkinsfile to Repository

In Ubuntu:

```shell
nano Jenkinsfile
```

👉 Add this:

```
pipeline {
    agent any

    stages {
        stage('Build') {
            steps {
                echo 'Building project...'
            }
        }
        stage('Test') {
            steps {
                echo 'Testing project...'
            }
        }
        stage('Deploy') {
            steps {
                echo 'Deploying project...'
            }
        }
    }
}
```

Push it:

```shell
git add Jenkinsfile
git commit -m "Added Jenkinsfile"
git push
```

✅ g. Configure Build Trigger

Option 1: Poll SCM (easy for exam)

In Jenkins job:

- Go to **Build Triggers**
- Enable:
  ✔ Poll SCM

Add:

```
None
* * * * *
```

👉 Checks every minute

Option 2: GitHub Webhook (advanced)

1. In GitHub repo → Settings → Webhooks
2. Add:

```
None
http://your-ip:8080/github-webhook/
```

✅ h. Push Code Changes

```
Shell
nano app.py
```

Change:

```python
print("Updated code for Jenkins build trigger")
```

Push:

```shell
git add .
git commit -m "Updated code"
git push
```

✅ i. Verify Automatic Build

👉 Go to Jenkins dashboard

✔ You will see:

- Build triggered automatically
- Click build → Console Output

👉 Output shows:

```
Building project...
Testing project...
Deploying project...
```

🎯 Final Output (Exam Checklist)

✔ GitHub repo created
✔ Jenkins installed & running
✔ Plugins installed
✔ Pipeline job created
✔ Jenkinsfile added

✔ Build trigger configured
✔ Code pushed
✔ Build triggered automatically

🧠 Important Concepts (Viva)

Q1. What is Jenkins?

👉 Automation server for CI/CD

Q2. What is Pipeline?

👉 Set of stages (build, test, deploy)

Q3. What is Jenkinsfile?

👉 Script defining pipeline

Q4. What is Poll SCM?

👉 Jenkins checks repo periodically

Q5. What is Webhook?

👉 GitHub notifies Jenkins instantly

⚠️ Common Mistakes

❌ Wrong repo URL
❌ Missing Jenkinsfile

❌ Jenkins not running
❌ Plugin not installed

🔥 Exam Tip

👉 Always show:

- Jenkins dashboard
- Running build
- Console output

👉 This gives full marks 💯

If you want, I can give you **short viva answers + diagram of CI/CD pipeline (very useful for theory)** 🚀

9.

Perfect 👍 — this is your **Docker practical on Ubuntu + Jenkins verification**. I'll give you a **clear, exam-ready step-by-step**.

🐳 ✅ PART 1: Verify Jenkins Auto Trigger (Short)

👉 After pushing code to **GitHub**:

1. Open Jenkins → http://localhost:8080
2. Go to your Pipeline job
3. Click **Build History**

✔ You should see a new build automatically triggered

👉 Click build → **Console Output**
✔ Shows pipeline stages (Build/Test/Deploy)

🐳 ✅ PART 2: DOCKER PRACTICAL (UBUNTU)

✅ a. Create Project Folder

```shell
mkdir docker-ai-app
cd docker-ai-app
```

Create `app.py`

```shell
nano app.py
```

👉 Code (simple Flask AI app):

```python
from flask import Flask

app = Flask(__name__)

@app.route('/')
def home():
    return "AI Model Running Successfully!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

Create `requirements.txt`

```shell
nano requirements.txt
```

👉 Add:

```none
flask
```

✅ b. Create Dockerfile

```shell
nano Dockerfile
```

✅ c. Write Dockerfile Instructions

👉 Paste this:

```none
# Base image
FROM python:3.9

# Set working directory
WORKDIR /app

# Copy files
COPY . .

# Install dependencies
RUN pip install -r requirements.txt
```

```
# Expose port
EXPOSE 5000

# Run application
CMD ["python", "app.py"]
```

✅ d. Build Docker Image

```shell
docker build -t ai-app .
```

👉 ✔ Image created

Check:

```shell
docker images
```

✅ e. Run Docker Container

```shell
docker run -d -p 5000:5000 ai-app
```

👉 `-d` → run in background
👉 `-p` → map port

✅ f. Verify Container is Running

```shell
docker ps
```

✔ You should see container running

✅ g. Test Application

👉 Open browser:

```none
http://localhost:5000
```

✔ Output:

```none
AI Model Running Successfully!
```

✅ h. Stop Container

```shell
docker ps
```

Copy container ID

```shell
docker stop <container_id>
```

🎯 Final Output (Exam Checklist)

✔ Project folder created
✔ app.py + requirements.txt
✔ Dockerfile created
✔ Image built
✔ Container running
✔ App working on localhost:5000
✔ Container stopped

🧠 Important Concepts (Viva)

Q1. What is Docker?

👉 Containerization platform

Q2. What is Dockerfile?

👉 Instructions to build image

Q3. What is Image?

👉 Blueprint of container

Q4. What is Container?

👉 Running instance of image

Q5. Difference VM vs Docker?

👉 Docker is lightweight, VM is heavy

⚠️ Common Mistakes

❌ Forgetting `EXPOSE 5000`
❌ Wrong port mapping
❌ Not installing dependencies
❌ Typo in Dockerfile

🔥 Exam Tip

👉 Always show:

```shell
docker ps
```

✔ Running container

👉 Then open browser → show output

👉 This gives full marks 💯

If you want, I can give you **Kubernetes (Minikube) next step or viva questions from Docker + Jenkins combined** 🚀

10.
Great 👍 — this is an **advanced Docker Compose practical** (multi-container AI app). I'll give you a **simple, exam-friendly setup on Ubuntu** that you can easily explain.

🐳 ✅ DOCKER COMPOSE PRACTICAL (MULTI-CONTAINER AI APP)

👉 We will create **3 containers**:

- AI Model (Flask app)
- Database (MongoDB)
- Frontend (simple HTML)

✅ a. Create Project Structure

```shell
mkdir ai-compose-app
cd ai-compose-app

mkdir backend frontend
```

✅ b. Backend (AI Model - Flask)

```shell
cd backend
nano app.py
```

👉 Code:

```python
from flask import Flask
from pymongo import MongoClient

app = Flask(__name__)

client = MongoClient("mongodb://db:27017/")
db = client.testdb

@app.route('/')
def home():
```

```
    return "AI Model Connected to Database!"

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

---

requirements.txt

```shell
nano requirements.txt
```

```
None
flask
pymongo
```

Dockerfile (backend)

```shell
nano Dockerfile
```

```
None
FROM python:3.9
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```

✅ c. Frontend (Simple Web Page)

```shell
cd ../frontend
nano index.html
```

👉 Code:

```html
<!DOCTYPE html>
<html>
<head>
    <title>AI App</title>
</head>
<body>
    <h1>Welcome to AI Application</h1>
    <p>Backend is running!</p>
</body>
</html>
```

Dockerfile (frontend)

```shell
nano Dockerfile
```

```
FROM nginx:latest
COPY index.html /usr/share/nginx/html/index.html
```

✅ d. Create docker-compose.yml

Go to main folder:

```shell
cd ..
nano docker-compose.yml
```

👉 Paste this:

```none
version: '3'

services:

  backend:
    build: ./backend
    ports:
      - "5000:5000"
    depends_on:
      - db

  db:
    image: mongo
    ports:
      - "27017:27017"

  frontend:
    build: ./frontend
    ports:
      - "8080:80"
```

✅ e. Build and Run All Containers

```shell
docker-compose up --build
```

👉 This will:

- Build backend & frontend images
- Pull MongoDB image
- Start all containers

✅ f. Verify Containers Running

```shell
docker ps
```

✔ You should see:

- backend
- db
- frontend

✅ g. Test Application

Backend:

👉 Open browser:

```none
http://localhost:5000
```

✔ Output:
`AI Model Connected to Database!`

Frontend:

👉 Open:

```
None
http://localhost:8080
```

✔ Shows HTML page

✅ h. Stop All Containers

```
Shell
docker-compose down
```

🎯 Final Output (Exam Checklist)

✔ Multi-container setup
✔ Backend (AI model) running
✔ Database connected
✔ Frontend working
✔ docker-compose.yml used
✔ All containers started together

🧠 Concept Explanation (Viva)

Q1. What is Docker Compose?

👉 Tool to manage multiple containers

Q2. Why use Docker Compose?

👉 Run multi-container apps easily

Q3. What is `depends_on`?

👉 Ensures service starts after another

Q4. How containers communicate?

👉 Using service name (e.g., `db`)

Q5. Difference Docker vs Docker Compose?

👉 Docker = single container
👉 Compose = multiple containers

⚠️ Common Mistakes

❌ Wrong folder structure
❌ Wrong port mapping
❌ Missing Dockerfile
❌ Typo in service names

🔥 Exam Tip

👉 Say this line:

> "Docker Compose helps orchestrate multiple containers like backend, frontend, and database in a single command."

👉 This gives extra marks 💯

If you want, I can also give you **Kubernetes (Minikube) version of this same project** or **diagram for viva explanation** 🚀

11.
Perfect 👍 — this is your **Kubernetes (Minikube) practical on Ubuntu**. I'll give you a **clear, step-by-step exam-ready guide** to deploy and scale your Dockerized AI app.

☸️ ✅ KUBERNETES WITH MINIKUBE (UBUNTU)

✅ a. Install Minikube & kubectl

Install kubectl

```shell
sudo apt update
sudo apt install -y kubectl
```

Install Minikube

```shell
curl -LO
https://storage.googleapis.com/minikube/releases/latest/minikube-
linux-amd64
sudo install minikube-linux-amd64 /usr/local/bin/minikube
```

Check:

```shell
minikube version
kubectl version --client
```

✅ b. Start Minikube Cluster

```shell
minikube start
```

👉 This creates a **local Kubernetes cluster**

Check:

```shell
kubectl get nodes
```

✔ You should see node = `Ready`

✅ c. Use Your Docker Image in Minikube

👉 Instead of Docker Hub, use Minikube Docker:

```shell
eval $(minikube docker-env)
```

👉 Now build your AI app image again:

```shell
docker build -t ai-app .
```

✅ d. Create Deployment

```shell
nano deployment.yaml
```

👉 Paste:

```none
apiVersion: apps/v1
kind: Deployment
metadata:
  name: ai-app-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: ai-app
  template:
    metadata:
      labels:
        app: ai-app
    spec:
      containers:
      - name: ai-app
        image: ai-app
        ports:
        - containerPort: 5000
```

Apply deployment:

```shell
kubectl apply -f deployment.yaml
```

✅ e. Create Service

```shell
nano service.yaml
```

👉 Paste:

```none
apiVersion: v1
kind: Service
metadata:
  name: ai-app-service
spec:
  type: NodePort
  selector:
    app: ai-app
  ports:
    - port: 5000
      targetPort: 5000
      nodePort: 30007
```

Apply service:

```shell
kubectl apply -f service.yaml
```

✅ f. Verify Deployment

```shell
kubectl get pods
```

✔ Pods running

```shell
kubectl get services
```

✔ Service created

✅ g. Access Application

```shell
minikube service ai-app-service
```

👉 Opens browser automatically

✔ Your AI app runs 🎉

✅ h. Scale the Application

```shell
kubectl scale deployment ai-app-deployment --replicas=4
```

Verify scaling:

```shell
kubectl get pods
```

✔ Now 4 pods running

✅ i. Stop Cluster

```shell
minikube stop
```

🎯 Final Output (Exam Checklist)

✔ Minikube installed
✔ Cluster started
✔ Docker image used
✔ Deployment created
✔ Service exposed
✔ App accessible
✔ Scaling demonstrated

🧠 Important Concepts (Viva)

Q1. What is Kubernetes?

👉 Container orchestration system

Q2. What is Minikube?

👉 Local Kubernetes cluster

Q3. What is Deployment?

👉 Manages pods and replicas

Q4. What is Pod?

👉 Smallest unit in Kubernetes

Q5. What is Service?

👉 Exposes application

Q6. What is scaling?

👉 Increasing number of pods

⚠️ Common Mistakes

❌ Image not found
❌ Wrong port
❌ Forgetting `eval $(minikube docker-env)`
❌ YAML indentation errors

🔥 Exam Tip

👉 Always show:

```shell
kubectl get pods
```

✔ Multiple pods (after scaling)

👉 This proves Kubernetes working 💯

If you want, I can give you **short viva answers + diagram of Docker → Kubernetes flow (very useful for theory)** 🚀